A Minimal Executable Proof for Multi-Language Contract Traceability

Werner Kasselman
Verivus OSS
werner@verivus.com

May 22, 2026

Abstract

This paper reports a deliberately small executable proof for a DAG-TOML contract: six "Hello, world!" implementations in Rust, Go, C, Java, TypeScript, and AWK are linked to one observable-output contract, one implementation DAG, one traceability file, one readiness gate, and one evidence matrix. The load-bearing contract requires the exact UTF-8 byte sequence `Hello, world!\n`, zero stderr bytes, and exit code 0. On the runner used for this paper, the witness harness reported five PASS outcomes, one SKIP for Java because `javac/java` was not on `PATH`, and zero FAIL outcomes. Two sidecar witnesses exercise narrower source-analysis claims: a convoluted Go rewrite hides the contiguous greeting literal but remains visible to sqry at the declared AST symbol and simple-edge level, while an indirect AWK rewrite uses a declared source profile because AWK is not in the repository's sqry-backed validator language set. The contribution is not a benchmark, a claim of general semantic equivalence, or a production assurance system. It is a compact, falsifiable artifact that shows how a contract, implementation graph, traceability chain, and review gate can be checked against executable witnesses.

1 Introduction

Software-engineering papers about artifacts are easiest to review when the claims, commands, and files line up directly. Artifact-review guidelines make the same point operationally: reviewers need enough structure to connect a paper's claims to executable scripts, source files, expected outputs, and limitations [4, 10]. The DAG-TOML repository provides such a structure for engineering agents and reviewers. This paper examines the smallest nontrivial instance in the repository: `examples/proof-hello-world`.

The example has one primary runtime contract. Each canonical implementation must write the exact bytes `Hello, world!\n` to stdout, write no bytes to stderr, and exit with code 0. The example also contains two source-analysis witnesses. The Go witness demonstrates that a specific rewrite can remove the plain greeting literal while leaving declared function symbols and simple graph edges visible to AST-aware tooling. The AWK witness demonstrates a fallback profile for an unsupported language: because the repository's sqry-backed symbol validator covers Rust, Go, TypeScript, and Java, not AWK, the AWK rewrite is checked by a narrower deterministic source-profile script.

This is intentionally a minimal proof, not a performance experiment. There is no dataset, no timing comparison, no statistical model, and no claim that all source rewrites can be reduced to author intent. The goal is to show that the proof pack is inspectable and falsifiable in a single sitting.

2 Artifact

The proof pack lives under `examples/proof-hello-world`. Its normative files are:

- `CONTRACT_DECLARATION.toml`: contracts C01 through C06.
- `IMPLEMENTATION_DAG.toml`: nine units, all tier 1.
- `TRACEABILITY.toml`: intent, requirement, implementation, code, test, and output chains.
- `REVIEW_READINESS.toml`: one gate for the witness pack.
- `EVIDENCE_MATRIX.toml`: five claims mapped to seven evidence artifacts.
- `run_all.sh`, `detect_semantic_rewrite.sh`, and `detect_awk_rewrite.sh`: executable witnesses.

The canonical implementations are `src/rust/hello.rs`, `src/go/hello.go`, `src/c/hello.c`, `src/java/Hello.java`, `src/typescript/hello.ts`, and `src/awk/hello.awk`. The rewrite fixtures are `src/go_convoluted/hello.go` and `src/awk_convoluted/hello.awk`.

3 Contract and Witnesses

Table 1 lists the six declared contracts and the local witness that supports each claim. C01 is the load-bearing runtime contract. C02 through C04 are narrow output constraints that depend on C01. C05 and C06 are source-analysis contracts with explicit non-claims.

Contract	Domain	Witness and scope
C01	Observable output	`run_all.sh`; exact stdout bytes `Hello, world!\n`, empty stderr, exit 0, with missing toolchains reported as SKIP.
C02	Encoding	Depends on C01; output must be ASCII bytes that are valid UTF-8 and not locale-dependent in the tested pipe context.
C03	No terminal markup	Depends on C01; byte-exact comparison rejects ANSI escapes and other extra output bytes.
C04	No BOM prefix	Depends on C01; byte-exact comparison rejects any non-`H` first byte, including UTF-8 BOM.
C05	Go AST rewrite detectability	`detect_semantic_rewrite.sh`; literal absence, C01 runtime satisfaction, declared Go functions, caller edge, and import edge.
C06	AWK source-profile detectability	`detect_awk_rewrite.sh`; literal absence, C01 runtime satisfaction, shared BEGIN/print intent profile, and rewrite markers.

Table 1: Contract-to-witness inventory.

4 Implementation DAG

The implementation DAG is a fan-in graph. Six independent layer-0 units prepare the canonical source entries. A layer-1 verification unit consumes those source artifacts, then builds or runs only the entries whose required toolchains are available and enforces C01 on each executed artifact. Two additional layer-0 units run the independent Go and AWK rewrite witnesses.

```
  U01 rust   \
```

```
U02 go      \
U03 c        \
U04 java      > U06 verify-contract-c01
U05 ts       /
U08 awk     /

U07 verify-semantic-ast-rewrite     (independent leaf)
U09 verify-awk-rewrite-detection    (independent leaf)
```

The checked computed values are validated by the repository's implementation-DAG validator:

- nine units;
- layer counts `{0: 8, 1: 1}`;
- entry points `U01, U02, U03, U04, U05, U07, U08, U09`;
- leaf nodes `U06, U07, U09`;
- critical path `U05 -> U06` with critical-path LOC 138.

5 Observed Execution

All commands in this section were run from the repository root on May 22, 2026. The Java toolchain was not available on the runner, so the Java implementation is not claimed to have executed in this run. The harness reports that condition as SKIP, not PASS.

Command	Exit	Observed result
`bashexamples/proof-hello-world/run_all.sh`	0	5 pass, 1 skip, 0 fail. Rust, Go, C, TypeScript, and AWK passed; Java skipped because `javac/java` was absent.
`bashexamples/proof-hello-world/detect_semantic_rewrite.sh`	0	8 pass, 0 skip, 0 fail. Go rewrite witness passed.
`bashexamples/proof-hello-world/detect_awk_rewrite.sh`	0	6 pass, 0 skip, 0 fail. AWK source-profile witness passed.
`python3validators/validate_implementation_dag.py...`	0	Implementation DAG validation passed.
`python3validators/validate_traceability.py...--check-paths-exist`	0	Traceability validation passed with 30 entities and path checks enabled.
`python3validators/validate_review_readiness.py...`	0	Readiness-gate, contract-declaration, and evidence-matrix files passed.
`python3validators/validate_ijb_conformance.py<file>`	1	FAIL for each proof TOML when run exactly without `-repo-root`; the validator requires `-repo-root` for instance files.
`python3validators/validate_ijb_conformance.py<file>--repo-root.`	0	PASS for all five proof TOML files when the repository root is supplied.
`python3validators/validate_code_symbols.py...`	0	8 supported symbols checked and matched; 4 entries skipped for unsupported languages.

Table 2: Observed command outcomes.

During preparation, the harness was audited for whether it truly checked the trailing newline byte.

The original shell comparison read stdout through command substitution, which strips trailing newlines. The witness scripts were therefore corrected to compare output files with `cmp` against an explicit `Hello, world!\n` byte stream and to check stderr by file size. The outcomes in Table 2 are from the corrected witnesses.

6 PASS, SKIP, FAIL, and MEASURED

The proof uses four result words in a deliberately narrow way.

- PASS means the relevant executable witness or validator exited 0 and reported that the checked condition held.
- SKIP means a declared check was not performed because a required toolchain was unavailable. SKIP is not evidence that the skipped implementation satisfies the contract.
- FAIL means the executable witness or validator exited nonzero or reported a violated condition. The IJB no-`--repo-root` invocations in Table 2 are preserved as FAIL.
- MEASURED is reserved for descriptive observations that are not pass/fail gates. This proof reports no benchmark or performance measurement.

7 Claim Audit

Table 3 maps the paper's claims to the evidence used. The distinction between direct observation and inference is important: for example, the source files directly show the implementations, while the run commands directly show only the implementations whose toolchains were available.

Claim	Evidence source	Status	Counterexample boundary
The proof pack is structurally complete for this example.	`IMPLEMENTATION_DAG.toml`, `TRACEABILITY.toml`, `CONTRACT_DECLARATION.toml`, `REVIEW_READINESS.toml`, `EVIDENCE_MATRIX.toml`; validators.	Directly observed.	Does not imply production readiness.
The available canonical implementations satisfy C01 on this runner.	`run_all.sh` corrected byte comparison and observed output.	Directly observed for Rust, Go, C, TypeScript, AWK; Java skipped.	A runner without those toolchains would produce more SKIPs.
The Go rewrite hides the literal but exposes declared AST structure.	`src/go_convoluted/hello.go` and `detect_semantic_rewrite.sh`.	Directly observed.	Does not prove arbitrary obfuscation resistance.
AWK uses a fallback profile because it is unsupported by the sqry symbol validator.	`docs/LANGUAGE-VALIDATORS.md`, `validators/validate_code_symbols.py`, and `detect_awk_rewrite.sh`.	Directly observed plus narrow inference.	Does not prove broad AWK AST similarity.

Claim	Evidence source	Status	Counterexample boundary
Artifact organization follows current review expectations.	ACM badging, POPL artifact guidance, arXiv TeX guidance, and SIGSOFT standards [4, 10, 2, 1].	Cited.	No formal artifact badge is claimed.

Table 3: Claim-to-evidence audit.

8 Related Work

The proof's source-analysis claims sit in a long line of program similarity and clone-detection work. Text and token methods are useful baselines but can be affected by formatting, naming, and local rewrites; tree-based and graph-based methods inspect program structure instead [15, 11, 13]. DECKARD represents programs through AST-derived structural vectors for scalable clone detection [9], while GumTree is a well-known AST differencing system for source changes [6]. Recent robustness work shows why plagiarism-hiding transformations must be stated carefully and why no single checker should be treated as a legal or semantic oracle [5].

The artifact framing follows research-artifact guidance rather than benchmark methodology. ACM distinguishes artifact evaluation, availability, and result validation [4]; POPL's artifact guidance asks authors to map scripts and source files to paper claims [10]; SIGSOFT's empirical standards emphasize more specific and technical review checklists for software-engineering research [1, 12]. The executable specification angle also has precedent in trace specifications, where formal specifications can be turned into executable models used as consistency checks and prototypes [8].

9 Threats to Validity

The validity structure follows common software-engineering reporting practice for case-study and empirical claims: separate what was measured from the causal or external claims one might be tempted to draw [14, 7].

Construct validity. The proof measures whether a tiny contract is represented and checked, not whether DAG-TOML is sufficient for a large assurance pipeline. The Hello-World contract is intentionally small, so the paper avoids generalizing from it to complex I/O, sandboxing, supply-chain integrity, or legal provenance.

Internal validity. The strongest internal risk was the shell newline issue described above. It was corrected before the reported run. A remaining risk is that the scripts depend on local toolchain behavior; the Java result is explicitly SKIP on this runner.

External validity. The proof covers six toy implementations and two hand-written rewrites. It does not show that the same approach scales to real services, unsafe languages, concurrency, platform- specific encodings, or adversarial obfuscation.

Conclusion validity. The results are categorical command outcomes, not statistical estimates. The correct conclusion is that the declared witnesses passed, skipped, or failed as reported on this runner. No benchmark, precision, recall, or legal conclusion follows.

10 arXiv and Artifact Packaging Notes

The paper package is intentionally plain: `article`, `pdflatex`, BibTeX, `natbib`, and standard packages. Official arXiv guidance says TeX submissions are processed automatically, authors must inspect the generated PDF, required figures and bibliography inputs must be included, and extraneous files such as logs, aux files, backup files, unused figures, and referee material should be removed from the source package [2]. arXiv currently supports TeX Live 2025 by default and TeX Live 2023 as a selectable option, with `pdflatex` among the supported processors [3].

For this paper, the artifact is already part of the repository. An arXiv source package should include only `main.tex`, `references.bib`, and any generated `main.bbl` if submitting with precomputed BibTeX output. The repository code can be linked from the paper or uploaded as ancillary material if desired, but the TeX source package should not include unrelated repository files.

11 Limitations and Non-Claims

This paper does not claim:

- general semantic equivalence between arbitrary programs;
- arbitrary obfuscation resistance;
- broad AWK AST similarity or parser-backed AWK analysis;
- copyright, licensing, or authorship conclusions;
- production readiness for real assurance workflows;
- performance, scalability, precision, recall, or benchmark results.

The exact claim is narrower: the repository contains a small multi-language proof pack whose declarations and executable witnesses can be checked, whose unsupported-language boundary is explicit, and whose observed outcomes are reproducible by running the listed commands in an environment with the same toolchains.

12 Conclusion

The Hello-World proof demonstrates the mechanics of contract-centered traceability with a small enough artifact that a reviewer can inspect every moving part. The primary runtime witness passed for five available toolchains and skipped Java because the required Java tools were absent. The Go AST witness and AWK source-profile witness passed. The structural validators passed, and the IJB validator's requirement for `-repo-root` was observed rather than hidden. The resulting artifact is not broad evidence about semantic equivalence or production assurance. It is a compact, executable proof that claims can be tied to contracts, DAG nodes, code paths, witnesses, and review gates.

References

[1] ACM SIGSOFT. Empirical standards for conducting and evaluating research in software engineering, 2026. URL `https://www2.sigsoft.org/EmpiricalStandards/`.

[2] arXiv. Submit TeX/LaTeX, 2026. URL `https://info.arxiv.org/help/submit_tex.html`.

[3] arXiv. TeX live at arxiv, 2026. URL `https://info.arxiv.org/help/faq/texlive.html`.

[4] Association for Computing Machinery. Artifact review and badging version 1.1, August 2020. URL https://www.acm.org/publications/policies/artifact-review-and-badging-current.

[5] Hayden Cheers, Yuqing Lin, and Shamus P. Smith. Evaluating the robustness of source code plagiarism detection tools to pervasive plagiarism-hiding modifications, 2021. URL https://arxiv.org/abs/2102.03997.

[6] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering*, pages 313–324, 2014. doi: 10.1145/2642937.2642982.

[7] Robert Feldt and Ana Magazinius. Validity threats in empirical software engineering research: An initial survey. In *Proceedings of the 22nd International Conference on Software Engineering and Knowledge Engineering*, pages 374–379, 2010.

[8] Daniel M. Hoffman and Richard T. Snodgrass. Trace specifications: Methodology and models. *IEEE Transactions on Software Engineering*, 14(9):1243–1252, 1988. doi: 10.1109/32.6168.

[9] Lingxiao Jiang, Ghassan Misherghi, Zhendong Su, and Stéphane Glondu. DECKARD: Scalable and accurate tree-based detection of code clones. In *Proceedings of the 29th International Conference on Software Engineering*, pages 96–105, 2007. doi: 10.1109/ICSE.2007.30.

[10] POPL 2023 Artifact Evaluation Committee. POPL 2023 artifact evaluation, 2023. URL https://popl23.sigplan.org/track/POPL-2023-artifact-evaluation.

[11] Lutz Prechelt, Guido Malpohl, and Michael Philippsen. Finding plagiarisms among a set of programs with JPlag. *Journal of Universal Computer Science*, 8(11):1016–1038, 2002. doi: 10.3217/jucs-008-11-1016.

[12] Paul Ralph, Sebastian Baltes, Domenico Bianculli, Yvonne Dittrich, Michael Felderer, Robert Feldt, Antonio Filieri, Carlo A. Furia, Daniel Graziotin, Peng He, et al. Empirical standards for software engineering research, 2020. URL https://arxiv.org/abs/2010.03525.

[13] Chanchal K. Roy, James R. Cordy, and Rainer Koschke. Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of Computer Programming*, 74(7):470–495, 2009. doi: 10.1016/j.scico.2009.02.007.

[14] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi: 10.1007/s10664-008-9102-8.

[15] Saul Schleimer, Daniel S. Wilkerson, and Alex Aiken. Winnowing: Local algorithms for document fingerprinting. In *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*, pages 76–85, 2003.